
Exercice 1 : Nettoyage et analyse de données (Niveau Facile)

Énoncé :

Tu reçois une liste de mails d'utilisateurs contenant des doublons, des espaces indésirables et des formats non homogènes (majuscules/minuscules).

1. Écris une fonction `nettoyer_emails(liste_emails: list[str]) -> set[str]` qui supprime les espaces superflus, passe tous les emails en minuscules et élimine les doublons.
2. La fonction doit retourner un ensemble (`set`) d'emails propres.

Données de test :

```
emails_bruts = [  
    "  Alice@Example.com  ",  
    "bob@example.com",  
    "ALICE@example.com  ",  
    " charlie@domain.fr",  
    "BOB@example.com",  
]
```

Exercice 2 : Gestion d'un inventaire de stock (Niveau Intermédiaire)

Énoncé :

Tu dois gérer l'inventaire d'une boutique représenté sous forme de dictionnaire : la clé est le nom du produit, et la valeur est un dictionnaire contenant le `prix` et la `quantite`.

1. Écris une fonction `calculer_valeur_stock(inventaire: dict) -> float` qui renvoie la valeur totale du stock ($\text{prix} \times \text{quantité}$ pour chaque produit).
2. Écris une fonction `filtrer_produits_en_rupture(inventaire: dict, seuil: int = 5) -> list[str]` qui renvoie la liste des noms des produits dont la quantité est strictement inférieure au seuil donné. Utilise une compréhension de liste.

Données de test :

```
inventaire = {  
    "Ordinateur": {"prix": 850.0, "quantite": 4},  
    "Clavier": {"prix": 45.0, "quantite": 15},  
    "Souris": {"prix": 25.0, "quantite": 2},  
    "Écran": {"prix": 210.0, "quantite": 8},  
}
```

Exercice 3 : Analyseur de texte avec `*args` et `**kwargs` (Niveau Avancé)

Énoncé :

Créer une fonction `analyser_texte(texte: str, *mots_cles: str, **options: bool) -> dict` qui :

1. Compte le nombre total de mots dans `texte`.
2. Compte le nombre d'occurrences de chaque mot clé passé dans `*mots_cles` (en ignorant la casse).
3. Si l'option `inclure_majuscules=True` est transmise dans `**options`, compte également le nombre de mots démarrant par une majuscule dans le texte.

Solutions et Corrections

Correction Exercice 1

```
def nettoyer_emails(liste_emails: list[str]) -> set[str]:
    # Utilisation d'une compréhension d'ensemble (set comprehension)
    return {email.strip().lower() for email in liste_emails}

# Test
emails_bruts = [
    " Alice@Example.com ",
    "bob@example.com",
    "ALICE@example.com ",
    " charlie@domain.fr",
    "BOB@example.com",
]

emails_propres = nettoyer_emails(emails_bruts)
print(emails_propres)
# Résultat : {'alice@example.com', 'bob@example.com', 'charlie@domain.fr'}
```

Explication : `strip()` retire les espaces avant/après et `lower()` normalise la casse. L'utilisation des accolades `{}` pour la compréhension génère directement un `set`, éliminant automatiquement les doublons.

Correction Exercice 2

```
def calculer_valeur_stock(inventaire: dict) -> float:
    # Somme des (prix * quantite) pour chaque produit
    return sum(
        item["prix"] * item["quantite"] for item in inventaire.values()
    )
```

```
def filtrer_produits_en_rupture(
    inventaire: dict, seuil: int = 5
) -> list[str]:
    # Compréhension de liste filtrée
    return [
        nom
        for nom, details in inventaire.items()
        if details["quantite"] < seuil
    ]

# Test
inventaire = {
    "Ordinateur": {"prix": 850.0, "quantite": 4},
    "Clavier": {"prix": 45.0, "quantite": 15},
    "Souris": {"prix": 25.0, "quantite": 2},
    "Écran": {"prix": 210.0, "quantite": 8},
}

valeur_totale = calculer_valeur_stock(inventaire)
ruptures = filtrer_produits_en_rupture(inventaire, seuil=5)

print(f"Valeur totale du stock : {valeur_totale:.2f} €") # 4245.00 €
print(f"Produits sous le seuil : {ruptures}") # ['Ordinateur', 'Souris']
```

Correction Exercise 3

```
def analyser_texte(
    texte: str, *mots_cles: str, **options: bool
) -> dict:
    mots = texte.split()
    total_mots = len(mots)

    # Nettoyage des mots du texte pour le comptage des mots-clés
    mots_lower = [m.strip(".,!?").lower() for m in mots]

    # Comptage des mots-clés demandés
    occurrences_clef = {
        mot: mots_lower.count(mot.lower()) for mot in mots_cles
    }

    resultat = {
        "total_mots": total_mots,
        "occurrences": occurrences_clef,
    }

    # Prise en compte des options optionnelles
```

```
if options.get("inclure_majuscules", False):
    mots_majuscule = sum(1 for m in mots if m[0].isupper())
    resultat["mots_majuscule"] = mots_majuscule

return resultat

# Test
phrase = "Python est un langage puissant. Python est simple et Python est rapide."
analyse = analyser_texte(
    phrase, "Python", "langage", "java", inclure_majuscules=True
)

print(analyse)
# Résultat :
# {
#   'total_mots': 12,
#   'occurrences': {'Python': 3, 'langage': 1, 'java': 0},
#   'mots_majuscule': 3
# }
```
